

DepthWizard Deck Guide & Defence Notes

Part 1 explains the presentation one slide at a time, and defines every technical word in plain English.
Part 2 is forty-three questions a judge could ask, ordered from the easiest to the hardest, each with an answer you can say out loud.

PROBLEM STATEMENT

SIH26175 — DepthWizard: Single-View Height Estimation and 3D Flythrough

THEME & CATEGORY

Disaster Management · Software

DECK

SIH2026-DevUp-SIH26175-DepthWizard-C · 6 slides

ORGANISATION

ISRO — Space Applications Centre

LIVE SITE

project5.zaidansari.tech

COMPILED

6 September 2026

CONTENTS

What is in this document

Read Part 1 once, so you know what sits on each slide. Read Part 2 twice.

FRONT MATTER

- Seven errors found and corrected
- The five things to remember

PART 1 – THE DECK, SLIDE BY SLIDE

- 1 Title slide
- 2 Problem statement
- 3 Technical approach
- 4 Feasibility and viability
- 5 Impact and benefits
- 6 Research and references

PART 2 – QUESTIONS, EASIEST TO HARDEST

- Q01 Before the questions start — your own script **(3)**
- Q04 Level 1 — Easy **(8)**
- Q12 Level 2 — Moderate **(11)**
- Q23 Level 3 — Hard **(12)**
- Q35 Level 4 — Hardest **(9)**

BACK MATTER

- Glossary — every term, in plain English

SEVEN ERRORS FOUND AND CORRECTED

I checked the deck against the official SIH portal page and against your actual code. All seven are already fixed in the deck you are presenting. They are listed here because a judge may be holding an older copy, and because knowing what was wrong is how you avoid saying it out loud.

1. **The theme is wrong.** Slide 1 says "Space Technology". The portal says **Disaster Management**.
2. **The problem title is not exact.** Yours says "Single-View Height Estimation and 3D Terrain Visualisation". The official one is **"DepthWizard - Single-View Height Estimation and 3D Flythrough"**.
3. **Team ID still says <fill from portal>** on slide 1.

4. **3.62 m is not the number your software actually produces.** That figure comes from a slightly different setup. The version running on your website scores **3.667 m**, which makes you **38%** better than the benchmark, not 39%. This appears on slides 1 and 6.
5. **React is listed but you do not use it.** It only appears inside the scripts that build the PowerPoint, as the name of an icon. Your viewer uses no framework at all.
6. **OpenCV is listed but you do not use it.** It appears nowhere in your code.
7. **It says Python 3.11.** You actually use 3.10.7.

THE FIVE THINGS TO REMEMBER

- 1 | **Your error is 3.667 m. The published benchmark is 5.9 m.** You are better by 38%.
- 2 | **The hard part is not running a depth model.** It is turning an answer with no units into an answer in metres.
- 3 | **You get tall buildings badly wrong.** Above 20 m you are about 17 m too short. Say this before a judge finds it.
- 4 | **You split your data by region, not randomly.** This makes your score worse but honest. Most teams do it the easy way and get a better-looking number.
- 5 | **Half the marks are for the 3D viewer.** Show the live website. Do not show a screenshot of it.

The deck, slide by slide

Every element on every slide: what it means, and the sentence to say when a judge points at it. If a judge points at anything on screen, the answer is in here.

SLIDE 1

Title and headline numbers

This slide has one job: make the judges decide you are worth listening to. It does that by leading with measurements instead of promises.

The name and tagline

DEPTH WIZARD – "One satellite image in, a measurable 3D surface out."

The important word is **measurable**. Not "accurate", not "high-precision". Measurable means someone can check it. That sets up everything that follows.

The details block

Problem ID, problem title, theme, category, team ID, team name. This is admin information, and SIH checks it matches their portal exactly.

Three of the seven errors listed above are in this block.

PS	Problem Statement. Every SIH problem has an ID number. Yours is SIH26175.
Category	Software or Hardware. Yours is Software.
Theme	The topic SIH files the problem under. Yours should say Disaster Management.
Dev Up	Your registered team name.

"BUILT AND MEASURED, NOT PROPOSED"

Most SIH teams present a plan for something they intend to build. This line says yours already exists and has been tested.

Every number underneath it is there to back up that claim.

The three boxes

These are not random statistics. They answer the three questions every judge is silently asking, in order.

BOX	QUESTION IT ANSWERS
3.62 m error per building	Is it accurate? Measured against a real laser scanner, not our own opinion.
39% better than 5.9 m	Is it better than what already exists? Yes, than a published dataset.
509 ms per tile, no graphics card	Can it actually be used? Yes, on ordinary hardware.

Each one alone proves nothing. Accuracy alone invites "better than what?". A comparison alone invites "on what hardware?". Together they close the argument.

RMSE	Root Mean Square Error. Square every mistake, average them, take the square root. A few big mistakes push this number up a lot.
Per building	Comparing one height per building against one true height, rather than comparing every single pixel. This is what published research does, so it is the only fair comparison.
Airborne LiDAR	A laser scanner flown on an aircraft. It measures distance by timing light pulses. This is your source of correct answers.
GlobalBuildingAtlas	A published worldwide dataset of building heights. Its error for Asia is 5.9 m, and that is the benchmark you beat.
Tile	A 518 by 518 pixel piece of the photo. Big photos are processed in overlapping pieces because the model only accepts that one size.

IF A JUDGE CHECKS THE MATHS

5.9 minus 3.62, divided by 5.9, gives 38.6%, which rounds to 39. That works.

But with your real shipped number of 3.667 it gives 37.8%, which rounds to 38. Change the number rather than arguing about rounding.

The colour strip along the bottom

"The palette is the model's own output scale: 0 m at the left, 155 m at the right."

The colours used throughout this deck are the same colours your software uses to show height.

Dark navy on the left means 0 m, which is ground level. The colours move through blues to amber and rust on the right, which is 155 m. That is the tallest building in your test data.

So the deck is not decorated with random brand colours. It is coloured using its own data. If a judge asks, that is the whole answer.

Problem and solution

Three problems, six answers, and three screenshots that prove the software runs.

"Every submission promises accuracy. This one reports it."

Same idea as the banner on slide 1. Keep repeating it. It is what separates you from teams presenting intentions.

Problem 1: Archives with no height

India has decades of satellite photos already taken by Cartosat. Every one of them is a single photo.

To get height the normal way you need two photos from different angles, or a laser survey. **You cannot go back in time and take a second photo.** So those old photos can never gain height information by the usual methods.

This is the strongest argument in the whole deck. It makes your approach the *only* option for that archive, not just a cheaper one.

Problem 2: Answers with no confidence

Guessing height from one photo has no single correct answer. Many different real-world scenes could produce the same flat image.

So if software gives you one confident number, it is hiding the fact that it might be badly wrong. That is why you also output how sure the model is.

Problem 3: Checkpoints are not products

Research papers usually publish a trained model file and a script to test it. That is useful to other researchers and useless to anyone else.

Someone doing actual work needs something they can open, look at, and check. This is your argument for why the 3D viewer matters — and remember, it is worth half the marks.

Cartosat	ISRO's series of Earth-observation satellites. Their photo archive is what this problem is about.
Checkpoint	A saved file containing a trained model.

The six things you built

CLAIM ON THE SLIDE	WHAT IT ACTUALLY MEANS
One image to a metric DSM	One photo in, a height map in real metres out, whether or not the photo knows its location.
Confidence, not just height	Every pixel also gets a "how sure am I" number.
A navigable 3D scene	You can fly around it in a browser, compare against real data, and see an error map.
Validation inside the product	The accuracy figures are shown for each landscape type, not hidden behind one flattering average.
Absolute heights, two ways	Two methods for turning heights into real elevations: a free global elevation map, or known reference points.
Deployable, not demo-ware	No graphics card needed, and the viewer is a single file you can open from a USB stick.

DSM	Digital Surface Model. A map of how high everything is, including buildings and trees.
Metric	In real units, meaning metres. The opposite of a result with no units.
GeoTIFF	An image file that knows where on Earth it was taken.
Sigma	The model's own statement of how unsure it is about a pixel, in metres.
Monotonically	Consistently, without going backwards. Here it means: the more unsure the model says it is, the more wrong it actually is, every time.
MAE	Mean Absolute Error. The plain average mistake.
Correlation	How well your answers follow the real pattern, from -1 to $+1$.
GCP	Ground Control Point. A place where the real height is already known, used to correct your results.
Power-law fit	A correction curve rather than a straight line. Used because your error grows with height instead of staying constant.
Demo-ware	Software that only works in a carefully staged demonstration. You are claiming the opposite.

The three screenshots

"THE PRODUCT, RUNNING – CAPTURES OF THE LIVE VIEWER, NOT MOCK-UPS"

- **Sikkim, India.** The ground rises and falls by 967 m across 2 km. This is your evidence for hilly terrain, and it is Indian.
- **Omaha, height above ground.** You can see flat roofs, tree tops and even kerbs. This shows fine detail.
- **Omaha, confidence.** This shows where the model is unsure.

EXPECT THIS QUESTION

"Are these real screenshots?" Yes, and the caption says so. They are captures of the live viewer, which is running right now at project5.zaidansari.tech. Offer to open it there and then.

Technical approach

The slide judges question hardest. It has four parts: how the software works, how it would be adopted, what you decided and why, and how you tested it.

How the software works, left to right



Input. Three possibilities: a photo that knows its location, a plain photo that does not, or an old Cartosat photo. The first gives real elevations. The second only gives shapes. That split is exactly what the problem statement asks for.

Normalisation. Work out how many metres each pixel covers, then resize the photo so each pixel covers 0.3 m.

Why this matters: the model cuts the image into 14 by 14 pixel squares. At 0.3 m per pixel, one square covers 4.2 m of ground — about part of a roof. If you feed it a blurrier photo at 0.6 m per pixel, each square now covers 8.4 m, so a small building gets mixed in with the ground around it and comes out too short. Resizing fixes that.

The model. Two parts. An encoder that understands the image, and a decoder that turns that understanding back into a detailed answer for every pixel.

Dual head. Two outputs per pixel: the height, and how unsure the model is.

Post-processing. Cut into 518 pixel squares, blend the overlaps, run each square several times for accuracy, then add ground elevation.

Output. A height map file, a confidence map, and the 3D scene in the browser.

ViT	Vision Transformer. A type of network that cuts an image into fixed squares and lets every square take account of every other square, instead of only nearby ones.
Encoder	The part that turns an image into an internal description of what is in it.
DPT	Dense Prediction Transformer. The part that turns a coarse grid of squares back into a full-detail answer for every pixel. "Dense" just means every pixel gets an answer.
Neck	The middle stage between the encoder and the final output.
Mu	The predicted value. Here, the height in metres.
Log-variance	A way of storing the uncertainty. Using the logarithm keeps the value positive automatically and makes training more stable.
Fine-tuned	Taking a model someone else trained and training it further on your own data.
Tiling	Cutting a big photo into overlapping small pieces and blending the joins so no seams show.
TTA	Test-Time Augmentation. Running the same piece through the model several times, flipped and rotated, then averaging. Slower but more accurate.
Scale anchor	The thing that turns "height above the ground" into "height above sea level".

The four adoption steps

1. **Start with data that is already free.** Run it over ISRO's freely available tiles and publish results for each landscape type rather than one average.
2. **Test against real Indian measurements.** If ISRO shares their own survey data, your Sikkim comparison becomes a real accuracy figure.
3. **Package it for their systems.** It ships as a self-contained package that their other software can call, and the viewer needs no server.
4. **Run it across the whole archive.** Process the back catalogue in bulk. Every existing photo gains a height layer without commanding a satellite to take anything new.

Bhoonidhi	ISRO's public data website. You are registered on it.
CartoDEM	ISRO's Indian elevation map. Free only to Indian government bodies, not to students.
LISS-4	A camera on ISRO's Resourcesat satellites, about 5.8 m per pixel.
SAC	Space Applications Centre in Ahmedabad. The ISRO centre that set this problem.
DGPS	A more accurate form of GPS, corrected using a fixed station on the ground. Accurate to centimetres, so it can be used as truth.
Container	Software packaged with everything it needs, so it runs the same way on any computer.
API	A way for other software to use your software automatically, instead of a person clicking buttons.
Tasking	Instructing a satellite to photograph a particular place. It costs money. Your point is that reprocessing old photos costs nothing extra.

Decisions, and why

Four claims, each backed by a measurement. The point of this block is to show your choices were tested, not guessed.

1. "The head was chosen by measurement. All three land at a 0.43–0.49 slope."

Here is what slope means. Draw a graph. Real height along the bottom, your predicted height up the side. Plot every building as a dot.

If the model were perfect, the dots would form a diagonal line rising at 45 degrees. That is a slope of 1.0.

Your dots form a flatter line, slope about 0.47. That means as buildings get taller, your predictions do not rise fast enough.

REAL HEIGHT	YOUR MODEL SAYS
5 m	about 5 m
20 m	about 12 m
100 m	about 50 m

You tried three different designs for the final part of the model. All three gave a slope between 0.43 and 0.49, so all three were equally affected. Since none was better, you kept the simplest.

2. "Uncertainty is calibrated." When the model says it is unsure, it really is wrong. Measured at +0.829, where 1.0 would be perfect.

3. "Export is checked, not assumed." You converted the model to a portable format, then ran 804,972 test inputs through both versions to confirm they gave the same answers.

4. "Resolution is handled explicitly." Your resizing step recovers all but 3.6% of the accuracy lost on blurrier photos.

ECE	Expected Calibration Error. Measures whether stated confidence matches real accuracy. Yours is 0.077; lower is better.
Rank correlation	Comparing the order of things rather than the values. +0.829 means the pixels flagged as least certain really are the most wrong.
Held-out	Data the model has never been trained on.
ONNX	A standard file format for neural networks. Lets the model run without installing PyTorch.
int8	Storing the model's numbers with less precision to make the file 63% smaller. Costs 0.161 m of accuracy.
Binned head	Predicting which height range a pixel falls into, instead of an exact number. Tried, no improvement.
Head-tail cut	A published technique for separating roof pixels from ground pixels. It works, but it was not what was holding you back.

How it was tested

THING	YOUR CHOICE
How the data was split	By geographic region, not randomly
How much was tested	80 photos, about 12 million pixels, 3,090 buildings
How results are reported	Separately for city, open land, forest and mixed, and by building height
What is kept back	A final test set that has never been scored

The region split is the one to be able to defend. If you shuffle photos randomly, neighbouring photos end up on both sides. They share the same streets, buildings and lighting. A model can then score well by memorising a neighbourhood instead of learning to judge height.

Splitting by whole region stops that. It makes your score worse and honest.

Technology list

Python 3.11 · PyTorch · Hugging Face · ONNX Runtime · NumPy / SciPy · OpenCV · GDAL / rasterio · three.js · React · Docker

PyTorch	The software framework the model is built and trained in.
Hugging Face	The website and library you download the pre-trained model from.
ONNX Runtime	The program that runs your converted model without needing PyTorch.
NumPy / SciPy	Libraries for doing maths on large grids of numbers.
GDAL / rasterio	Libraries that read and write map image files and handle coordinate systems.
three.js	The library that draws the 3D view inside a web browser.
Docker	A way of packaging software with everything it needs so it runs anywhere.

THREE OF THESE ARE NOT TRUE

React only appears in the scripts that build this PowerPoint, as the name of an icon. Your viewer uses no framework.

OpenCV does not appear anywhere in your code.

Python 3.11 should say 3.10.7.

Remove the first two and correct the third. Saying "every dependency free and open" invites a judge to ask you to show them one.

Feasibility and viability

This is where you admit your weaknesses. That sounds bad but it is a strength: a weakness you measured and explained looks like good engineering.

The four claims across the top

Technically proven · Operationally simple · Economically viable · Socially useful

Standard SIH structure. Yours are each backed by something real: it is built and measured, it needs one photo and no special hardware, everything it uses is free, and height data is genuinely used in planning, disaster response and forestry.

Accuracy across the four landscapes

The problem statement specifically asks for stable performance across city, open, hilly and forested areas. So this block answers a requirement you are scored on.

LANDSCAPE	ERROR	SHAPE MATCH
Open land	1.837 m	+0.583
Mixed	2.723 m	+0.824
Forest	3.319 m	+0.780
City	13.860 m	+0.808

The slide's wording is good: three of four are under 3.4 m, and the fourth is one specific identifiable problem. You name the weakness and immediately narrow it down.

EXPECT A QUESTION ABOUT HILLS

Hills are missing from this table because your training data has no hills. Jacksonville and Omaha are both flat. You cover hills using Sikkim imagery, but there is no laser data for Sikkim, so it is a visual check rather than a score. Say this before you are asked.

Breaking down the city error

"79% of squared error comes from 1.9% of buildings. Under 10 m – 94% of the stock – we are at 1.4 m."

This is the most important sentence on the slide.

It works because of how RMSE is calculated. Every mistake is squared before averaging, so a mistake of 20 m counts a hundred times more than a mistake of 2 m. A small number of badly wrong tall buildings can dominate the whole figure.

You are not making the error disappear. Those tall buildings really are wrong by about 17 m. You are showing that you know exactly which buildings fail and why.

The two real risks

Risk 1: no Indian ground truth. Your only Indian check compares against Google's building height data, and finds you 6.94 m lower on 168 buildings.

The slide states clearly that this is one estimate compared against another estimate, not a measurement of accuracy. You also quote the worse of the two available figures on purpose — the other one looks better but is less fair.

Risk 2: tall buildings come out too short. This is a data problem, not a design problem. Your training photos only go up to 82.8 m, while your test photos reach 155.3 m.

Two possible fixes are named: a correction curve using known reference points, which cuts the error on tall buildings by 24.9%, and properly, getting training data with tall buildings in it.

Bias	Whether you are consistently too high or too low, rather than randomly wrong.
Footprint	The outline of a building seen from above.
Compress toward the mean	Everything gets pulled toward the average. Tall things come out too short, very short things slightly too tall.
Google Open Buildings	A Google dataset of building outlines and heights, worked out from much blurrier satellite photos. Useful as a comparison, never as truth.

Reducing the India risk before you have Indian data

RISK	WHAT YOU DID ABOUT IT
Their photos are blurrier	Automatic resizing recovers all but 3.6%
Their camera angle differs	Your data already spans 4.8 to 28.9 degrees, so the effect can be measured on data you have
Indian terrain is steeper	Tested on 31-degree slopes. The model does not confuse slope with building height.
Licensing could block deployment	Everything you use is free and open

Off-nadir	How far the camera is tilted away from looking straight down. At 0 degrees it looks straight down. At 20 degrees you can see some building sides.
Leakage	Here it means the model accidentally mistaking a steep hillside for a tall building. You tested for it and it does not happen.

Impact and benefits

Who would use this and why it is worth funding. The least technical slide, but it has the abbreviations most likely to catch you out.

Who it serves

WHO	WHAT THEY GET
ISRO and SAC	Heights from photos they already have, with no new satellite passes
Urban planning	Building heights for checking legal limits and modelling shadows
Disaster response	A surface map straight after an event, without waiting for a survey
Forestry	Tree heights, used to estimate how much wood and carbon a forest holds
Telecom	Checking whether two towers can see each other, without a laser survey

FSI	Floor Space Index. The total floor area of a building divided by the size of its plot. Indian city rules limit it, so checking building heights checks whether the rule was followed.
AMRUT	An Indian government scheme that funds city infrastructure improvements. Mentioned because it pays for exactly this kind of city data work.
Shadow study	Working out what a new building will block sunlight from. Needs the heights of surrounding buildings.
Canopy height	How tall the tree layer of a forest is.
Biomass	The total weight of living plants in an area. Estimated from tree height, and used for carbon accounting.
Encroachment	Illegal building or tree cutting on protected land. Visible as a change in height.
Line-of-sight	Whether two points can see each other in a straight line. Needed for radio links between phone towers, and blocked by buildings and hills.

The four numbers

NUMBER	WHAT IT MEANS
1 image	You need one photo, not a pair
0 graphics cards	Half a second per tile on a normal processor
36.8 MB	The whole model, small enough to work offline
4 landscapes	Scored separately, never hidden in one average

Quantised, if it appears, means the model's numbers are stored with less precision to make the file smaller. That is what gets it down to 36.8 MB, and it costs 0.161 m of accuracy.

The closing line

"The cost avoided is the second acquisition, not the first."

This is the sharpest money argument in the deck.

You are not claiming to make satellite photos cheaper. You are saying: the photo has already been taken and already been paid for. You get a completely new product out of it without spending anything more.

That is much harder to argue with than a claim about being cheaper.

Research and references

Proof that you know the field. Judges scan this to see whether you actually read the papers or just listed them.

Research that changed what you built

PAPER	WHAT YOU TOOK FROM IT
Depth Anything V2 (2024)	The model you started from
HTC-DC Net (2023)	A technique for separating roof pixels from ground pixels
Depth Any Canopy (2024)	The method for adapting a depth model to aerial photos
Beta-NLL (2022)	A fix that stops the height output from stalling during training
IM2HEIGHT, TSE-Net	Earlier attempts at this same problem, used as your comparison point

NeurIPS	The biggest machine learning conference in the world.
TGRS	The leading journal for satellite and remote sensing research.
ICLR	Another major machine learning conference.
et al.	Latin for "and others". It means the paper had more authors than the one named.
Beta-NLL	An adjustment to how the model is trained. Without it, teaching the model to report uncertainty can accidentally stop it improving its height estimates.
Baseline	Earlier work that you measure yourself against.

Tried, measured, rejected

This is the most convincing block in the deck. Anyone can list what worked. Listing what failed, with the number that killed it, is what separates real testing from marketing.

IDEA	WHY IT WAS DROPPED
Predict a height range instead of a number	No improvement
Measure shadows to get height	Even done perfectly it scores 0.503. Your model already scores 0.787.
Make training focus harder on tall buildings	The assumption was wrong. Tall pixels are 7.5% of the data but already cause 78.8% of the error.
Average two models together	Both models make almost identical mistakes, so averaging changes nothing.
Rescale all outputs to undo the squashing	Moves the error around instead of removing it.

Oracle	A pretend perfect version of something, used to find out the best it could possibly do. "Oracle shadows" means: even with flawless shadow measurement, this method only reaches 0.503, which is worse than what you already have. So it cannot help.
Photogrammetry	Measuring real-world sizes and distances from photographs.
LDS	A published technique for making training pay more attention to rare cases. Rejected because the assumption behind it did not hold for your data.
Ensemble	Averaging several models together. Only helps when they make different mistakes. Yours make the same ones.

The probes

A **probe** is a small experiment where you wrote down what you expected, and what result would prove you wrong, *before* running it.

PROBE	WHAT IT FOUND
01 Tall buildings	Not fixable with more training. The training data only goes up to 82.8 m.
03 Sharpening	Trying to square off roofs made every measurement worse.
04 Missing detail	Detail is limited by how much ground each square covers.
05 Resolution	Blurrier ISRO-style photos cost only 3.6% after automatic resizing.
06 Height squashing	Measured exactly: <code>ours = 0.473 × truth + 2.66 m</code>

Probe 06 is the one to understand properly. That formula lets you predict what your model will say for any building.

Take a 100 m building. Multiply by 0.473 to get 47.3. Add 2.66. You get about **50 m**.

Take a 5 m building. Multiply by 0.473 to get 2.37. Add 2.66. You get about **5 m**, which is correct.

So your entire weakness is described by one line. It also proves the error is predictable rather than random.

If a judge asks why you do not just divide by 0.473 to fix it — that is the "rescale" idea in the rejected list above. It does not work, because the model is not uniformly shrinking everything. It genuinely cannot tell a 26 m building from a 76 m one, and no amount of rescaling creates information that is not there.

The comparison table

Seven rows, three columns: a laser or stereo survey, other researchers' single-photo work, and yours. Each row is marked yes, partly, or no.

You lose two of the rows. A laser survey beats you on those two. You left them in deliberately.

THE BEST LINE IN THE DECK

"Two rows the alternatives win. A stereo survey gives absolute height directly and ships mature viewers — the table is only useful to a panel if the rows we lose are still in it."

The reasoning: if a comparison table shows you winning every single row, judges assume you chose the rows to flatter yourself. Nobody believes it. By showing the two rows where you lose, the five where you win become believable.

The benchmark, repeated

3.62 m per-building RMSE against airborne LiDAR, versus 5.9 m published for Asia. Scored on 3,090 buildings across 80 held-out tiles.

Same number as slide 1, so the same correction applies. Your real shipped figure is **3.667 m**. Fix both slides together.

Deck file: `SIH2026-DevUp-SIH26175-DepthWizard-C.pptx`. Every figure was checked against your own measurement records, the official SIH portal page, and your source code. Where the deck disagreed with the evidence, it is flagged above.

Questions, easiest to hardest

Forty-three questions in four levels of difficulty. Level 1 is what everyone asks. Level 4 is what a judge asks when they want to find out whether you really built it.

PART 2 · 1 OF 5

Before the questions start

How to open, how to run the demo, and what to do if the internet fails. These are not judge questions. They are your own script.

Q01 What do you say in the first thirty seconds?

SAY THIS

"To measure the height of buildings from space, you normally need two photos of the same place taken from different angles. Or a laser scanner flown on a plane. Both are slow and expensive."

"We do it with one ordinary photo. Our software turns it into a 3D map of building and ground heights, and you can fly through that map in a web browser. It takes about a minute, and it runs on a normal computer with no special hardware."

That covers four things at once. What the problem is. How you solve it. What you built. And the fact that it runs anywhere.

Q02 In what order should you run the demo?

1. **Open a scene that is already loaded.** The Jacksonville one looks good. Do not start by uploading a new image. Uploading takes 60 seconds and you would be standing in silence.
2. **Fly around it.** Show that the buildings have real shape, and that the original photo is wrapped over the 3D surface.
3. **Turn on the truth overlay.** This shows your result next to real laser-scanner data. Very few teams can show this. It is your strongest moment.
4. **Turn on the confidence layer.** Say: "the model marks where it is unsure, and when it says it is unsure, it really is wrong."
5. **Now start an upload** and keep talking while it runs.

DO NOT DO THIS

Do not invite a judge to upload any image they like as your first move. If they pick a photo of a city centre, that is your worst case. You would be explaining a weakness before you have shown anything good.

Q03 What if the website is down, or the Wi-Fi fails?

Carry the file `viewer_standalone.html` on a USB stick.

It is one file, 15.2 MB. You double-click it and the whole 3D viewer opens in a browser. No internet. No server. No installation.

You tested this properly. You did not just look at the file and assume it worked. You actually opened it in a browser and checked that the 3D view appeared.

This file also earns marks on its own. The problem statement asks for software that can be deployed on its own, and this is exactly that.

PART 2 • 2 OF 5

Level 1 — Easy

Almost every judge opens with one of these. If you cannot answer them without thinking, nothing further matters.

Q04 What exactly did ISRO ask for?

Software that takes one satellite photo and produces a height map. It has to handle two kinds of input.

INPUT	WHAT YOU MUST PRODUCE
A plain photo (PNG or JPG) with no location information	A relative height map. Shapes are right, but there are no real units.
A photo that knows where on Earth it was taken (GeoTIFF)	An absolute height map, in real metres.

Then you have to wrap that photo over a 3D surface, show it in a browser, and let someone fly around it and measure heights and slopes.

Marks are split **50% for accuracy, 50% for the 3D viewer**. Remember that. Half the marks are for something that is not the model at all.

Q05 Why does disaster management need heights?

Three concrete reasons.

- **Floods.** Water flows downhill. To predict where it goes, you need to know the shape of the ground.
- **Landslides.** Risk depends on how steep a slope is and how much material sits on it.
- **Damage assessment.** If you know building heights before and after an event, you can see what collapsed.

The catch is timing. Booking a laser survey or a special satellite pass takes weeks. After a cyclone you do not have weeks. Your method uses a photo that already exists, and gives an answer in about a minute.

Walk me through your pipeline.

Seven steps, in order.

1. **Someone uploads a photo.** PNG, JPG or GeoTIFF.
2. **You check the scale.** If the photo knows where it was taken, you can work out how many metres each pixel covers. You then resize the photo so each pixel covers 0.3 m, which is what your model was trained on.
3. **You cut it into squares.** The model only accepts 518×518 pixel images, so a big photo is cut into overlapping squares.
4. **Each square goes through the model.** For every pixel it outputs two numbers: a height, and how confident it is about that height.
5. **You glue the squares back together** and blend the overlaps so no seams show.
6. **If the photo has location data,** you add ground elevation underneath, so heights become real elevations above sea level.
7. **You build the 3D scene.** The height numbers become a 3D surface, and the original photo is painted onto it. Then it renders in the browser.

What model is underneath? Did you train it yourself?

You started from **Depth Anything V2 Small**. It is a free, openly licensed model that estimates depth from a single photo.

But you did not just download it and use it. **You retrained it** on aerial photos with real measured heights. That retraining is what turns it from "this is nearer than that" into "this is 12.4 metres tall".

IMPORTANT

Never say "we used a pre-trained model" and stop there. It makes it sound like you downloaded something and ran it.

Say instead: "we took a pre-trained model and retrained it on aerial height data, which is what makes it output real metres."

One more useful fact. This model comes in three sizes: Small, Base and Large. Only **Small** has a licence that allows commercial use. Base and Large are non-commercial only. So Small is the only one ISRO could actually deploy.

Q08 What is your accuracy?

MEASUREMENT	VALUE
Typical error per building	3.667 m
Average error per building	1.498 m
Half of all buildings are within	0.894 m
Are you too high or too low on average?	0.237 m too low

For context: half the buildings in your test set are under **4.22 m** tall.

Two words you need to know.

RMSE means root mean square error. You take every mistake, square it, average them, then take the square root. The squaring means a few big mistakes push the number up a lot. Your RMSE is 3.667 m.

MAE means mean absolute error. This is just the plain average mistake, ignoring whether you were too high or too low. Your MAE is 1.498 m.

MAE is always smaller than RMSE. When the gap between them is big, it tells you a few large mistakes are dominating. Yours is 1.498 versus 3.667, so yes — a few bad ones are dragging it up. Those are the tall buildings.

Q09 Is 3.667 m good? Compared to what?

There is a published dataset called **GlobalBuildingAtlas**. It covers building heights worldwide. Its published error for Asia is **5.9 m**.

You are at 3.667 m. That is **38% better**.

You also work from much sharper photos than they do — 0.3 m per pixel against their 3 m.

SAY THIS

"Our benchmark was GlobalBuildingAtlas, which publishes 5.9 m error for Asia. We are at 3.667 m."

Q10 Is it actually deployed, or just running on your laptop?

It is live at **project5.zaidansari.tech**, hosted on Microsoft Azure. Anyone can open it right now on a phone. No login.

It has six ready-made 3D scenes, and working upload. An upload takes about **60 seconds** from photo to finished 3D scene.

Q11 Does it need a graphics card?

No. The live website runs on an ordinary processor with no graphics card at all.

Processing one 518×518 square takes **509 milliseconds** — about half a second.

This matters more than it sounds. No graphics card means no expensive hardware to buy, a few dollars a month to run instead of hundreds, and it works on whatever computer a field office already has.

Level 2 – Moderate

Technical, but expected. A judge who has read the problem statement properly will ask most of these.

Q12 Why is this hard? Why not just use an existing depth model?

This is the most important question in the whole document. Here is the answer in three steps.

Step 1. Depth models have no units. A depth model tells you which things are closer and which are further away. It does not tell you by how much. A building could be 5 m tall or 50 m tall, and the model's output looks exactly the same.

Step 2. They learned from photos taken by people on the ground. Think of a normal holiday photo. It has a horizon. It has a road that gets narrower as it goes away from you. It has sky at the top. It has people and cars whose size you already know. All of these are clues about scale.

Step 3. Satellite photos point straight down. Looking straight down, there is no horizon. Nothing gets narrower with distance. There is no sky. Every clue the model learned to use has disappeared.

ISRO says this themselves in the problem statement. Their words: *"Converting relative depth into metric elevation remains a critical challenge."*

SAY THIS

"Normally you measure height by comparing two photos taken from different angles. That is how your eyes judge distance. With only one photo you cannot do that, so the usual method for getting scale is simply not available. That is the problem we solved."

Q13 What did you change inside the model?

Three changes.

1. You made it produce two answers instead of one. Originally it output one number per pixel: the depth. You kept that, and added a second output next to it: how uncertain the model is about that pixel. This is what powers your confidence layer.

2. You removed a function called ReLU from the end. ReLU forces any negative number to become zero. The original model needed that, because depth can never be negative. But it causes a problem during training: once a pixel gets pushed below zero, it gets stuck there and stops learning forever. You removed it and let the training handle it instead.

3. You made it output metres. The original model produced numbers around 1. Building heights are in the tens of metres. Rather than force the model to learn that big jump, you let it keep producing small numbers and multiply the result by a fixed factor. The benefit is that your reported error is directly in metres, which is exactly what ISRO scores.

A NICE DETAIL TO MENTION

"Depth Anything outputs larger numbers for things that are closer to the camera. When you are looking down from space, closer means taller. So the model's existing behaviour already pointed the right way. We were adjusting something that already made sense, not fighting it."

Q14

What data did you train on?

A dataset called **DFC2019**. It contains aerial photos of **Jacksonville and Omaha** in the United States, at 0.3 m per pixel, with real heights measured by an aircraft-mounted laser scanner.

Why that one? Three reasons. It has real measured heights to train against. It is free. And it is the dataset other researchers use, so your results can be compared to published work.

Your testing used **80 photos, about 12 million pixels, and 3,090 buildings**.

Q15

How do you get from "no units" to real metres? This is the core question.

There are **two separate conversions** here. People mix them up. Keep them apart.

Conversion 1: from "no units" to metres above the ground.

You did not do this with a clever formula afterwards. You did it by **retraining the model** on photos where the real heights were known. After enough examples, the model simply learned to output metres. That is the whole reason retraining was necessary instead of using the model as-is.

Conversion 2: from "metres above the ground" to "metres above sea level".

Your model says how tall something is compared to the ground right underneath it. A 10 m building on a 100 m hill gives you 10.

But ISRO asked for a surface model, which is height above sea level. For that building you need 110. So you add the ground elevation:

THE FORMULA

height above sea level = ground elevation + our height above ground

The ground elevation comes from **Copernicus GLO-30**. It is a free worldwide elevation map. It is rough — one value every 30 metres — but that is fine, because you only need it for the ground underneath, and the fine detail comes from your model.

BE HONEST ABOUT THIS WEAKNESS

Copernicus GLO-30 was made using radar. Radar bounces off the tops of buildings and trees, not the bare ground. So in a dense city, that "ground elevation" already includes part of the buildings.

Which means when you add your building height on top, you count some of it twice.

A proper fix needs a different dataset that is not free for commercial use. You have an option that reduces the problem approximately, and you describe it as approximate rather than pretending it is solved. Saying this yourself makes you look careful, not weak.

Q16 How does the model know when it is wrong?

Start with why this matters. Guessing height from one photo has no single correct answer — many different 3D scenes could produce the same flat photo. So giving one confident number would be dishonest.

Instead, your model gives two numbers per pixel: **a height, and how unsure it is.**

The clever part is how it is trained. The training rule lets the model reduce its penalty by admitting uncertainty — but admitting uncertainty costs it something too. So it cannot just claim to be unsure about everything to avoid being marked wrong. It has to be honest.

MEASUREMENT	VALUE	WHAT IT MEANS
Calibration error	0.077	When it says it is 80% sure, it is right about 80% of the time. Lower is better.
Uncertainty vs actual error	+0.829	The places it flags as unsure really are the places it gets wrong.
Typical stated uncertainty	1.082 m	On average it says "give or take about a metre".

That **+0.829** is the useful one. It means someone using your maps can trust the confidence layer to tell them which areas need checking.

Q17 Your measuring tool shows a "plus or minus" figure. Where does that come from?

This answer will impress a technical judge, so it is worth learning.

When you measure the height difference between two points, you want to say how uncertain that measurement is. The obvious way is to combine the uncertainty of both points using the standard formula.

But that formula assumes the two points are wrong *independently* of each other. Yours are not.

You measured what actually happens. Two pixels one metre apart are wrong together **94% of the time**. Their errors only become independent when they are about **60 metres apart**.

Think of it this way. If the model reads a whole rooftop 3 m too low, then measuring from one corner to the other still gives the right answer, because both ends are wrong by the same amount. The error cancels out.

So the standard formula gives an uncertainty about **3.5 times too large** for measurements on one roof. You measured the real relationship and corrected for it.

Q18

Why do you report error per building instead of per pixel?

Because per-pixel error is misleading at building edges.

Picture the edge of a roof. On one side of the line the correct answer is 20 m. On the other side it is 0 m. If your model puts that line even slightly in the wrong place, those few pixels are wrong by the entire height of the building.

That inflates the number enormously and it is not comparable to anything. Your per-pixel building error is 16.256 m. Your per-building error is 3.667 m. Same model, same data — just a different question.

Every published paper reports one height per building. So do you.

MENTION THIS YOURSELF. IT EARNS TRUST.

When you compare a building, you take the middle value of your prediction across that roof, and the middle value of the truth. Middle against middle.

You could cheat here by taking a high value of yours against the middle of theirs. That would make your numbers look 1.3 m better. You tested that, saw it was misleading, and did not do it.

Q19

How did you split your data for testing, and why does it matter?

You split by **region**. Whole areas of a city go into training, or into testing, never both.

Here is why that matters. The easy way is to shuffle all the photos randomly and take some for testing. But neighbouring photos overlap in style: same streets, same architecture, same lighting, sometimes the same buildings.

So a model can score well just by memorising a neighbourhood, without ever learning to judge height.

Splitting by region blocks that. It makes your score **worse but trustworthy**.

You also have a third set of data you have never scored at all. It is kept for the very end, and it is not even stored on the training computer.

Q20

ISRO asks for stability across landscapes. Show me.

LANDSCAPE	ERROR	SHAPE MATCH
Open land	1.837 m	+0.583
Mixed	2.723 m	+0.824
Forest	3.319 m	+0.780
City	13.860 m	+0.808

Three out of four are strong. City is four to seven times worse. That is because cities are where tall buildings are, and tall buildings are your weakness.

But look at the last column. "Shape match" stays high in the city at +0.808. That means the model still puts the tall buildings in the right places and gets their relative order right. It just gets the actual numbers too small.

Right shape, wrong scale. That is a much better position to be in than being randomly wrong.

Q21 What is the ONNX export and why does it exist?

ONNX is a standard file format for neural networks. Saving your model in this format means it can run without installing PyTorch, which is a large and complicated piece of software.

	FULL PRECISION	COMPRESSED
File size	100.7 MB	36.8 MB
Biggest difference from the original	0.0010 cm	0.161 m

You did not assume the converted model still worked. You ran **804,972 test inputs** through both versions and compared. The full-precision one matched to a hundredth of a millimetre.

The compressed version stores numbers with less precision to save space. It is 63% smaller but costs 0.161 m of accuracy. Say that openly — it is a genuine trade-off, and a judge will check.

One honest limitation: the exported model only accepts exactly 518×518 images. It cannot handle other sizes. That is fine because your software always feeds it 518×518, but you should say so rather than imply flexibility you do not have.

Q22 Why Three.js and not Unity?

The problem statement allows Three.js, Unity or Babylon.js.

Three.js runs **in a web browser with nothing to install**. A judge opens a link and it works. Unity would need a separate download and a different build for each operating system.

Since "can be deployed on its own" is directly worth marks, no installation wins.

It also made your single-file version possible: the entire viewer packed into one HTML file that runs from a USB stick with no server at all.

PART 2 · 4 OF 5

Level 3 — Hard

These go looking for the weak spot. Answer them by naming the weakness first, then saying what you did about it. Never let a judge find it before you have said it.

Q23 Why not use India's own elevation data, CartoDEM?

CartoDEM would be the obvious Indian choice. But it is free only to Indian government bodies.

Your college is a private institution, so you are not one, and you would have to pay for it.

That is a licensing restriction, not a technical opinion. Just say it plainly.

Q24 **ISRO will test on their own photos, at a different resolution. Will that break your model?**

It would, if you did nothing. And you measured exactly how badly.

First, why resolution matters. The model chops the image into small squares of 14×14 pixels and reasons about those squares. At 0.3 m per pixel, one square covers **4.2 m of ground**. That is roughly part of a roof.

Give it a photo at 0.6 m per pixel instead, and one square now covers **8.4 m**. A small building and the ground around it get averaged into one square. The result is that buildings come out too short.

PHOTO RESOLUTION	ERROR, DOING NOTHING	ERROR, AFTER YOUR FIX
0.3 m (what you trained on)	1.350 m	—
0.6 m	2.260 m	1.399 m
1.0 m	3.138 m	1.657 m

The fix is simple. Read from the photo's own data how many metres each pixel covers. Then enlarge the photo so each square covers 4.2 m of ground again. At 0.6 m this recovers almost all the lost accuracy — you end up within **3.6%** of never having lost it.

SAY THIS

"The problem statement says final testing uses ISRO's own satellite imagery, so we treated the resolution difference as a real risk and measured it, rather than hoping it would be fine."

Q25 **Why is forest worse than open land? Is your model bad at trees?**

No. This one is physics, not a flaw in your model.

The "correct answers" you compare against came from a laser scanner. Laser pulses slip through gaps in the leaves and hit the actual ground below. So the truth data knows where the real ground is.

A camera cannot do that. Leaves block the view completely. Your model can only see the top of the canopy.

So you are not making a mistake. You are being compared against information that is physically impossible to see in a photograph.

The proof is that the same model gets 0.87 m on open ground a few hundred kilometres away.

Q26 **What about hills? That is one of the four landscapes ISRO named.**

Answer honestly: **your training data has no hills**. Jacksonville and Omaha are both flat.

You cover hills using imagery from Sikkim, in the Indian Himalayas. But there is no laser data for Sikkim, so you cannot calculate a real accuracy figure. You report it as a visual check, not a score.

You did test one specific fear. On steep ground, does the model confuse the slope with building height? On 31-degree slopes, the relationship between predicted height and ground steepness is **-0.044**, which is effectively zero. No confusion.

Q27 You have no Indian ground truth. So what does your India work prove?

Be careful and precise here.

There is no laser data for your Indian scenes. So you compared against a different source: Google's building height dataset, which is itself estimated from much blurrier satellite images.

That means you are comparing **two estimates against each other**. Neither is truth. You must say that.

SIKKIM	DIFFERENCE
All 585 buildings	1.15 m lower than Google
Only the 168 Google is confident about	6.94 m lower than Google

QUOTE THE WORSE NUMBER

Use **6.94 m**, not 1.15 m.

The reason: when Google is unsure about a building outline, that outline spills onto the surrounding ground and trees. Those sloppy outlines drag the comparison towards zero and make you look better than you are.

The 168 buildings Google is confident about give the fairer comparison, and it is the less flattering one. Quoting it yourself is what makes a judge trust the rest of your numbers.

Q28 Where does your error actually come from?

REAL HEIGHT	HOW MANY	ERROR	TOO HIGH OR LOW
0–3 m	166	1.282 m	slightly high
3–6 m	2,305	1.429 m	slightly high
6–10 m	443	1.947 m	slightly low
10–20 m	116	4.034 m	1 m low
over 20 m	60	23.451 m	17 m low

LEARN THIS SENTENCE

"79% of our total error comes from 1.9% of the buildings. Below 10 metres, which is 94% of all buildings, our error is 1.4 metres."

Q29

List all your limitations.

1. **Buildings over 20 m come out about 17 m too short.** Not enough tall buildings in the training data.
2. **Forest ground is 2.5 m off**, because a camera cannot see through leaves.
3. **The edges of each processed square are less accurate** than the middle, roughly double the error. This is a known behaviour of this type of model.
4. **The ground elevation data you add includes some building height already**, so in dense cities you count part of it twice.
5. **You have no laser data for India**, so Indian results are comparisons, not accuracy.
6. **You have a final test set you have never scored**, deliberately, until the very end.

Q30

Your viewer stretches heights by 1.5×. Isn't that cheating?

No, and you can prove it three ways.

- It only stretches the **picture**, not the data. The saved height file records no stretching at all.
- The measuring tool **removes the stretch** before it reports any number.
- The real laser data gets stretched by the same amount, so any comparison stays fair.

The slider is labelled on screen and says "the numbers stay true".

Why it exists. A typical scene covers about 300 m of ground but buildings are only 5 to 30 m tall. So the heights are tiny compared to the width. Without stretching, the 3D view looks almost completely flat and you cannot see anything. Drag the slider to 1.0 to see true proportions.

THE BEST FOLLOW-UP

"And stretching cannot be hiding anything, because we predict buildings too *short*, not too tall. On our city test image the tallest real building is 93.6 m and we say 33.9 m. Even stretched by 1.5, it still appears shorter than it really is."

Q31

Why not use shadows? That is how it was done before computers.

Good question, and you have a real answer.

The old method is simple: measure how long a building's shadow is, work out where the sun was, and the geometry gives you the height. It works best on **tall** buildings, which are exactly where your model struggles. So it sounds perfect.

You have the data needed — 67 files recording the sun's position for your images.

You even confirmed the shadows are clearly visible. On the tallest building in one image, walking away from the sun, the brightness drops from 166 on the roof to about 100 in the shadow, holds for 120 pixels, then jumps to 254 on sunlit ground.

But you measured its best possible performance and it still loses. Even assuming perfect shadow measurement, the method reaches a shape-match of 0.503. Your model already achieves 0.787.

So adding it would make things worse. It stays in your plan as a documented idea with evidence, rather than something you built.

Q32 Why not just use stereo photos or a laser scanner?

Because that is the whole point of the problem statement.

ISRO's own wording calls those methods *"cost-prohibitive, dependent on specific sensor availability, and computationally intensive"*.

If you already have stereo photos, you do not need this. The point is producing usable heights when you do not have them — which includes every photo already sitting in an archive.

Q33 Can I break your website?

There are limits in place, because each upload uses real computing time.

- **50 MB maximum file size.** TIFF, PNG or JPG only.
- **Maximum image size of about 13.8 million pixels.**
- **Damaged or half-downloaded files are rejected immediately**, by checking the file header rather than discovering the problem mid-processing.
- **Only three jobs at once**, and one upload per person every 20 seconds.

The pixel limit is the interesting one. File size does not predict how long processing takes — the number of pixels does. A satellite image can be 17408×17408 pixels (303 million pixels) in a file of only 12 MB, because it compresses well. That would take about 2.8 hours.

GOOD LINE

"A file size limit does not protect the server, because processing time depends on pixels, not megabytes. So we reject based on how long the job would actually take, calculated from measured speed."

Q34 What is TTA, and why isn't your best score the one you ship?

TTA means test-time augmentation. You run the same image through the model several times — flipped and rotated — and average the results. It takes longer but is more accurate.

SETUP	PER BUILDING	TOO LOW BY
Basic	3.771 m	0.31 m
With TTA	3.616 m	0.34 m
What you ship	3.667 m	0.24 m

Notice that what you ship is **0.051 m worse** than your best score. That is deliberate.

The shipped version adds a second pass at higher detail and blends it in. It costs 1.4% of accuracy. In exchange, edges are **28% sharper** — buildings look like proper flat-topped blocks instead of rounded lumps.

WHY THIS IS THE RIGHT CHOICE

"Half the marks are for the 3D view, so we accepted a 1.4% accuracy cost for a 28% improvement in how buildings look. And we report the numbers from the version we actually ship, because what we ship should be what gets scored."

Level 4 — Hardest

Questions designed to make you defend the whole idea. Every one of them has an honest answer, and the honest answer is stronger than a dodge.

Q35 Why can't you fix the tall buildings? Just train it more.

You tested that. It is a **data problem, not a training problem**.

Your training photos contain only 166 buildings over 30 m, and the tallest is **82.8 m**. But your test set contains buildings up to **155.3 m**.

So you are asking the model to judge something it has essentially never seen.

You tried training harder on the few tall buildings you have. It made things **worse**, not better. The model got better at those specific buildings and worse at new ones. That is the classic sign of learning examples by heart instead of learning the pattern.

Here is the detail that settles it. The model puts **91% of its confidence above 20 m** for a tall building. So it knows the building is tall. It simply cannot tell whether it is 26 m or 76 m.

THE STRONGEST WAY TO SAY THIS

"We tested every option inside the model. A bigger model. Different output designs. Different ways of reading the output. Combining two models. Changing the training to focus on tall buildings. None of them helped."

"What that leaves is a conclusion about the data, not the model: looking straight down at a roof, the photo does not contain enough information to tell 26 metres from 76 metres. Fixing it needs training data with tall buildings in it."

Q36

Have you tried... (the full list)

IDEA	WHAT HAPPENED
Use a bigger model	Size is not the problem. Every version tested squashes tall buildings by about the same amount.
Predict a height range instead of a number	No improvement.
Read the output differently	No hidden tall answer to recover. For 40 m+ buildings the model's most likely answer is 26 m when the truth is 76 m.
Average two models together	Pointless. Both models make almost identical mistakes, so averaging changes nothing.
Make training focus harder on tall buildings	The assumption was wrong. Tall pixels are 7.5% of the data but already cause 78.8% of the error, so training was already paying attention to them.
Sharpen the output to square off roofs	Made every measurement worse.
Calibrate each scene using known reference points	A trade, not a fix. Better on tall buildings, worse on average, and it made 7 of 13 test images worse.

SAY THIS

"Two of those five experiments proved us wrong about something we believed. We wrote down our prediction before running each one, which is why we treat these as findings rather than excuses."

Q37

ISRO recommends the GAMUS dataset. Why didn't you use it?

This is a fair hit, so have the answer ready.

GAMUS is the dataset ISRO's own page recommends. You trained on DFC2019 instead.

The honest reasons: DFC2019 is the same kind of data — sharp aerial photos with laser-measured heights — and it is the one published research uses. That is what lets you compare yourself to the 5.9 m benchmark. If you had used GAMUS, that comparison would not exist.

You have started loading GAMUS but have not finished.

TURN IT INTO A STRENGTH

"GAMUS is our next training set, and specifically because its cities are dense urban areas. That is exactly where our error is worst. So it targets the weakness we have already measured."

Q38 Isn't this just Depth Anything with extra steps?

No. Here is the precise difference.

Depth Anything used straight out of the box, on a satellite photo, gives you a shape with no units and poor accuracy, because everything it learned about scale came from ground-level photos.

What you added:

- Retrained it on aerial photos with real measured heights, so it outputs metres
- Added a second output that reports how confident it is
- Added ground elevation so results become real heights above sea level
- Added automatic resolution matching so it survives a different camera
- Built the entire 3D viewer and measuring tools

The original model is the starting point, not the solution.

Q39 Your city error is 13.9 m. So it doesn't work in cities — where disasters happen?

This is the sharpest question you will get. Answer in three parts.

First, agree. Cities are our weakest case, and the error is concentrated in buildings over 20 m.

Second, narrow it. 94% of buildings are under 10 m, and there our error is 1.4 m. The problem is a small, identified group. And the shape-match stays at +0.808, meaning we still place tall buildings correctly and rank them correctly — we just report them too short.

Third, point at the confidence layer. The model marks these exact areas as uncertain. So a person using the map knows which buildings to double-check.

FINISH WITH THIS

"A system that is wrong and knows it is very different from a system that is wrong and confident. That is why the confidence output was the first thing we built, not the last."

Q40 You tested on two American cities. Why should ISRO believe it works in India?

Do not overclaim. Structure the answer like this.

- Real accuracy figures only exist where laser data exists, which is Jacksonville and Omaha.
- For India you have a comparison against another estimate, and you report the less flattering of the two available numbers.
- The specific risk of a different camera resolution was measured and fixed, recovering all but 3.6%.
- The risk of steep Himalayan terrain confusing the model was tested and ruled out.

Then say what would close the gap: Indian height measurements to test against, or training data from dense cities.

Naming what you still need is a strength, not an admission.

Q41 How do we know you didn't just tune everything to your test set?

Three answers.

- **The region-based split.** No area appears in both training and testing, so memorising a neighbourhood cannot help.
- **A third set of data that has never been scored once.** It is deliberately kept off the training computer.
- **You wrote down your failed experiments.** A team that was tuning to look good would only report successes. Two of your five recorded experiments proved you wrong.

SAY THIS BEFORE THEY ASK

That untouched final test set contains only **one building over 30 m**. Since tall buildings are your weakness, that set will make you look better than you really are.

Whenever you quote a score from it, say that in the same breath.

Q42 Who built this? Can you explain code you didn't write yourself?

Answer straight. Most of the build is one person's work with AI assistance. That is normal now and is not a weakness. The weakness would be not understanding what was built — which is what this document exists to prevent.

IF YOU DON'T KNOW SOMETHING

Say so, then say where the answer is. For example: "I would need to check our calibration results for the exact figure."

That is a far better answer than a confident wrong number. Judges forgive gaps. They do not forgive bluffing, and an ISRO panel will catch it immediately.

Q43 What would you do with three more months?

Have this ready. It shows you know exactly where you stand.

1. **Get training data with tall buildings.** This is the one measured problem you have not solved. GAMUS first, because its cities are dense.
2. **Get Indian height measurements,** so your India results become real accuracy instead of a comparison.
3. **Finish the reference-point tool in the viewer.** The calculation already works and is tested; only the click-to-place interface is missing.
4. **Use shadows for tall buildings only.** Shadows are best exactly where your model is worst. It cannot replace the model, but it might correct the specific cases that fail.

Glossary

Every abbreviation and technical term used in the deck or in the answers above.

Every term you might be asked about, in plain English.

DSM	Digital Surface Model. A map of how high everything is, including buildings and trees. This is what ISRO asked for.
DEM	Digital Elevation Model. A general term for height data.
DTM	Digital Terrain Model. A map of the bare ground, with buildings and trees removed.
ⓇDSM	Relative DSM. The shape of the surface, but with no real units. What you produce for a photo with no location data.
AGL	Above Ground Level. How tall something is compared to the ground right underneath it. A 10 m building on a 100 m hill is 10 m AGL.
GSD	Ground Sample Distance. How much ground one pixel covers. 0.3 m GSD means each pixel is 30 cm across. Smaller means sharper.
Nadir	Looking straight down. This is what satellites usually do, and it is why depth models struggle — there is no horizon or perspective to judge scale from.
Off-nadir	Looking at an angle instead of straight down. At 17 degrees off-nadir you can see some of the sides of buildings.
LiDAR	A laser scanner, usually flown on an aircraft. It measures distance by timing how long a light pulse takes to bounce back. This is your source of correct answers.
Stereo	Using two photos of the same place taken from different angles to work out height. The normal method, and the one you are replacing.
Monocular	Using one image only. "Mono" means one.
Ill-posed	A problem with more than one possible correct answer. Many different 3D scenes could produce the same flat photo, which is why reporting uncertainty is the honest approach.
RMSE	Root Mean Square Error. Square every mistake, average them, take the square root. Because of the squaring, a few big mistakes push this number up a lot.

MAE	Mean Absolute Error. The plain average mistake. Always smaller than RMSE. A big gap between the two means a few large mistakes dominate.
Bias	Whether you are consistently too high or too low. Yours is 17 m too low for buildings over 20 m.
Correlation	How well your predictions follow the real pattern, from -1 to $+1$. High correlation with bad RMSE means the shape is right but the scale is wrong. That is your city situation.
Calibration	Whether the model's stated confidence matches how often it is actually right.
Sigma (σ)	The model's own statement of how unsure it is about a pixel, in metres.
Neural network	A program that learns patterns from examples instead of following written rules.
Backbone	The part of the model that understands the image. Almost all the learned knowledge lives here — 89% of yours.
Head	The small final part that turns understanding into the answer you want. Yours is about 0.1% of the model, and it is the part you modified.
Vision transformer (ViT)	A type of network that cuts an image into small squares and lets every square take account of every other square. Yours uses 14×14 pixel squares.
DINOv2	The specific backbone inside Depth Anything. It learned from images without any human labelling, which is why it works on many different kinds of picture.
DPT	Dense Prediction Transformer. The part that turns a coarse grid of squares back into a full-detail answer for every pixel.
Fine-tuning	Taking a model someone else trained and continuing to train it on your own data. This is what makes your model output metres.
Zero-shot	Using a model with no extra training at all. Doing this on satellite photos does not work well, which is why you fine-tuned.
Checkpoint	A saved file containing a trained model.
Overfitting	When a model memorises the training examples instead of learning the general pattern. It scores well on data it has seen and badly on anything new.
Region-disjoint split	Splitting data by geographic area, so no place appears in both training and testing. Stops the model scoring well by memorising a neighbourhood.

TTA	Test-Time Augmentation. Running the same image through several times, flipped and rotated, then averaging. Slower but more accurate.
Sliding window	Processing a large photo as overlapping small squares, because the model only accepts a fixed size.
ONNX	A standard file format for neural networks. Lets your model run without installing PyTorch.
Quantisation	Storing the model's numbers with less precision to make the file smaller. Yours saves 63% of the size and costs 0.161 m of accuracy.
GCP	Ground Control Point. A place where the true height is already known, used to correct a scene.
SRTM	A free worldwide elevation map made from a Space Shuttle mission in 2000. Mentioned in the problem statement.
Copernicus GLO-30	The free European worldwide elevation map you use instead. Same 30 m detail as SRTM, no account needed, and it covers the far north where SRTM has gaps.
CartoDEM	ISRO's Indian elevation map. Free only to Indian government bodies, so not available to you.
DFC2019	Your training dataset. Aerial photos of Jacksonville and Omaha at 0.3 m per pixel, with laser-measured heights.
GAMUS	The dataset ISRO's own page recommends. Covers Philadelphia, New York and Washington DC — all dense cities.
Georeferenced	A photo that knows where on Earth it was taken. GeoTIFF is the usual format.
GeoTIFF	An image file with location information built into it.
Megapixel	One million pixels. This is what determines processing time — not the file size.
Mesh	The 3D surface built from your height numbers.
Texture drape	Painting the original photo onto that 3D surface so it looks real.
Vertical exaggeration	Stretching heights in the 3D view so you can see the shape. Yours starts at 1.5× and is removed before any measurement is reported.

All figures come from the exact configuration running on the live website, tested on 80 photos, about 12 million pixels and 3,090 buildings, against aircraft laser measurements. Where no laser data exists, results are described as comparisons rather than accuracy.